Final System-Level Synthesis

During this project, we implemented and evaluated models across five paradigms: content-based filtering, item-item collaborative filtering, matrix factorization (ALS, FunkSVD), pairwise learning-to-rank (BPR), deep learning (NCF, MultVAE), ranking heuristics (popularity, graph propagation, Personalized PageRank), and hybrid approaches. We also designed an A/B test and ran bandit simulations to explore online evaluation. This section takes a broader view of our findings and asks: based on what we observed, what should we deploy, how can we improve it over time, and what risks should we consider?

Offline vs Online Discrepancies

The table below summarizes the final test performance of all models we implemented, ordered by NDCG@10.

Rank	Model	NDCG@10	Precision@10	Recall@10	Coverage
1	Hybrid (CF + TF-IDF, α=0.8)	0.1493	0.1360	0.0861	,
2	CF Item-Item (Cosine)	0.1488	0.1380	0.0865	~3%
3	CF Item-Item (Pearson)	0.1291	0.1160	0.0609	,
4	BPR (10 neg samples)	0.1178	0.0888	0.1066	19.3%
5	NCF (NeuMF)	0.1099	0.0855	0.0858	,
6	MultVAE	0.1015	0.0805	0.0816	,
7	Item-Item Graph	0.0884	0.0729	0.0631	3.3%
8	ALS (Implicit)	0.0844	0.0665	0.0701	,
9	Personalized PageRank	0.0782	0.0657	0.0524	3.1%
10	BPR (1 neg sample)	0.0774	0.0641	0.0556	19.3%
11	Popularity	0.0762	0.0642	0.0505	3.1%
12	FunkSVD	0.0404	0.0338	0.0247	,
13	CB (Jaccard)	0.0339	0.0320	0.0131	,
14	CB (TF-IDF)	0.0306	0.0290	0.0170	,

We used NDCG@10 on a temporal test split as our main metric for offline evaluation. This serves as a reasonable indicator of recommendation quality, but our experiments showed it can be misleading.

Popularity looks good on paper but would be unacceptable in practice

The popularity baseline scored NDCG@10 = 0.0762, surpassing FunkSVD (0.0404) and both content-based models. However, it recommends only about 110 unique items, which is 3.1% of the catalog. In a real system, showing all users the same list of popular items would quickly lead to user boredom. Its high score comes from the test set having a similar power-law distribution as the training data (slope -1.48 on a log-log scale).

Most test interactions involve popular items, so a model that always predicts popular items gets many "hits" for free.

Cold-start users are hidden in aggregate numbers

38% of test users did not appear in training. The overall NDCG for any model combines warm-user performance (where collaborative filtering excels) and cold-user performance (where most models score near zero). We did not break down our metrics by warm and cold users, so we cannot quantify how much of CF's 0.1488 NDCG comes from strong warm-user ranking versus how much is dragged down by cold users. A model with slightly lower overall NDCG but better cold-start performance may lead to more engaged users over time, since cold users are the most likely to leave.

Offline and online metrics disagree on model ranking

Collaborative Filtering (CF) outperformed MultVAE by 47% on NDCG@10 (0.1488 vs 0.1015). However, in the bandit simulation, the gap shrank to 1.3% on hit rate (6.02% vs 5.94%).

Setting	CF	MultVAE	Relative Gap
Offline NDCG@10	0.1488	0.1015	47%
Bandit Hit Rate	6.02%	5.94%	1.3%

This occurs because the metrics differ in how they operate: NDCG@10 is sensitive to the rank of relevant items, while hit rate is binary, either the held-out item appears in the top-10 or it does not. Hit rate is more forgiving, so some of the gap shrinkage is due to switching to a more lenient metric, not just the change in evaluation conditions. The evaluation settings are also different (bandit pulls happen sequentially over simulated interactions, one user at a time). The key point is that both the choice of metric and evaluation setup influence which model appears best, and relying on a single offline number can be misleading.

Coverage is invisible to ranking metrics

Item-item CF recommends a similar narrow slice of the catalog as the popularity baseline (around 3% coverage). BPR achieves 19.3%. NDCG does not reflect whether users with niche preferences are being served well or are just receiving the same popular movies as everyone else. In an active system, low coverage compounds over time: the recommender continues to show the same items, users interact with those items, and the model grows even more confident in recommending them.

Deployment Choice and Justification

We would deploy **item-item CF with cosine similarity** as the primary recommender, supported by a **popularity fallback for cold users** and a **content-based fallback for cold items**.

Why CF over the hybrid

The weighted hybrid (CF + CB, alpha=0.8) recorded NDCG@10 = 0.1493 compared to CF's 0.1488, a difference of 0.0005, which is negligible. The cascade hybrid (CF candidate generation + CB reranking) dropped to 0.0891 when evaluated alone. This drop does not indicate that cascading is a poor structure; it is widely used in production for good reasons. The issue is that our content features (an 18-genre taxonomy and TF-IDF on titles) are too broad to effectively rerank candidates that CF selected well. When the CB reranker

adjusts a strong collaborative ranking based on weak metadata, quality decreases. With richer content signals (plot summaries, cast, user reviews), cascading could enhance results. But with our current features, the slight improvement from blending does not justify maintaining two models during inference. CF is also easier to interpret: when a recommendation seems off, we can see which items in the user's history influenced it. With a hybrid, the mixed score complicates this.

Why not the deep learning models

NCF (0.1099) and MultVAE (0.1015) both perform 26-32% worse than CF on NDCG@10. Two factors contribute to this. First, with only 3,706 items, item-item CF can compute exact similarity across full interaction vectors, there's no need to compress through a learned bottleneck, so no information is lost. Second, neural models require a lot of data to learn representations that generalize beyond mere memorization. On a small catalog, k-NN-style memorization of co-occurrence patterns is effective, while neural architectures often struggle with overfitting or fail to develop useful abstractions. For example, MultVAE's variational bottleneck needs dense user profiles for effective reconstruction, something many users in this dataset lack. This limitation is specific to this dataset's scale. In a catalog with hundreds of thousands of items, where the full similarity matrix becomes impractical and enough interaction data is available for training, learned embeddings would probably be necessary, and model rankings could change.

Why not BPR as the primary model

BPR with 10 negative samples reaches NDCG@10 = 0.1178 and 19.3% catalog coverage, much more diverse than CF. However, when we examined performance by item popularity, BPR essentially fails on less popular items.

Model	Segment	NDCG@10	Recall@10
Popularity	head (top 20%)	0.0699	0.0573
Popularity	tail (bottom 80%)	0.0000	0.0000
BPR (uniform)	head (top 20%)	0.0857	0.0772
BPR (uniform)	tail (bottom 80%)	0.0007	0.0005

BPR's tail NDCG of 0.0007 is practically zero. Its coverage figure is valid, but it mainly personalizes recommendations within the popular group. This is a direct consequence of uniform negative sampling: since ~80% of items are in the tail, random negatives are almost always tail items, so the model learns to push them down rather than distinguish among them. BPR is better suited as a diversity supplement, mixing in a few candidates from BPR into the CF list instead of serving as the sole ranker.

Handling cold start

New users without interaction history will receive popularity-based recommendations. Once they provide a few ratings, we will switch to CF. For new items lacking collaborative signals, we will default to content-based scoring using TF-IDF on genres and titles. This maintains the same two-tier structure we tested in the hybrid experiments, candidate generation (CF) for warm cases, and metadata-based fallback for cold cases.

Iteration Strategy Post-Deployment

Our goal is to start simple and only introduce complexity when we have clear evidence it adds value.

Step 1: Launch and observe

We will deploy CF with the popularity fallback and log everything, which items are recommended, which ones are clicked, and which are skipped. We will establish a baseline click-through rate before making any adjustments. No retraining will occur during this phase. As shown in the online evaluation report, retraining with data influenced by the model's own recommendations creates feedback loops that can skew the evaluation.

Step 2: Test diversity

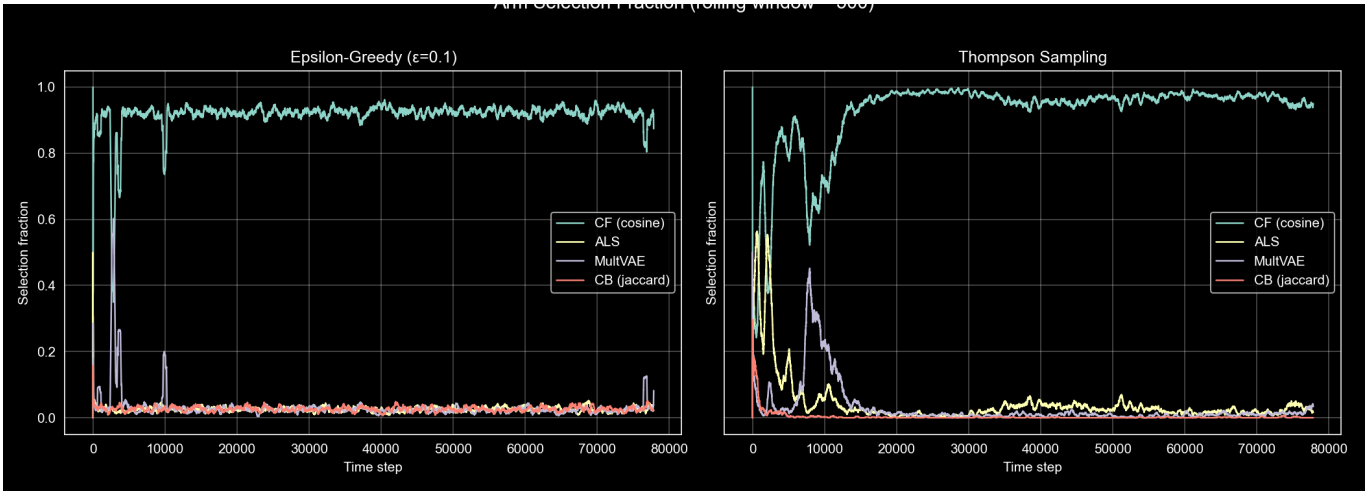
CF's biggest weakness is its low catalog coverage. We will conduct an A/B test where the treatment group receives a few BPR-generated candidates mixed into the lower positions of their recommendation list (for example, positions 7-10). This approach allows us to verify if users engage with more diverse content without impacting the top recommendations, where most clicks occur. If engagement improves while click-through rate remains steady, we will maintain the mix.

Step 3: Use bandits for model selection

Instead of committing to a single model permanently, we will implement Thompson Sampling across various candidate strategies (pure CF, CF + BPR diversity, MultVAE).

Strategy	Total Hits	Hit Rate
Static (CF)	4,689	6.02%
Thompson Sampling	4,635	5.96%
Epsilon-Greedy (ε=0.1)	4,547	5.84%

Our bandit simulation showed that Thompson Sampling converges to about 93% allocation for CF while still providing meaningful exploration for MultVAE (2,285 pulls) and largely dismissing clearly weak options like CB (206 pulls).



This method is more efficient than epsilon-greedy, which, in our experiment, allocated the same exploration budget to CB as it did to MultVAE, even though CB performed 10 times worse. One caution: our simulation used a stationary bandit (fixed posteriors that only grow more confident). In a real system where model

performance fluctuates due to new content, seasonal trends, or changes in the user base, we would need a non-stationary variant with decaying posteriors so the bandit can adapt rather than sticking to early results.

Step 4: Retrain and watch for drift

The CF similarity matrix is a 3,706 x 3,706 matrix that fits in memory and recomputes quickly. Regular retraining as new interactions come in keeps recommendations fresh. In our dataset, average ratings remained stable at 3.58 over the 2.8-year collection period, so temporal drift was not an issue. However, a live system would likely not be so stable, seasonal trends, new releases, and viral moments could all change user behavior, making regular recomputation essential.

Key Failure Modes to Monitor

Popularity collapse

CF already suggests about the same percentage of the catalog as the popularity baseline, approximately 3%. In a live system with feedback loops, popular items get recommended, gain more clicks, and appear even more popular. This situation can worsen. Track the number of unique items recommended each day. If that number keeps shrinking, diversity injection, such as using BPR candidates or re-ranking for variety, becomes necessary.

Cold-start churn

New users experience the weakest recommendations, which are based only on popularity. If they leave before interacting with enough items for CF to personalize their experience, the system loses them for good. Monitor the first-session click-through rate and the 7-day return rate for new users separately from the overall numbers. If retention for cold users is significantly lower, invest in a better onboarding flow. For example, ask users to select a few favorite genres or movies to kickstart their profile.

Stale similarity matrix

The item-item matrix is like a snapshot. Items added after the last recomputation are not visible to the recommender. If the catalog grows quickly, a larger share of new content may never get shown. Content-based fallback helps cover the gap between recomputations, but it is a much weaker signal. The frequency of recomputation should match the rate of new content.

Feedback loops

This is the most subtle failure mode. The recommender shapes the data it later uses for training. Our bandit simulation avoided this by freezing the models, but a real system cannot do that. Over time, the model becomes increasingly confident about a narrow set of items it has already shown. Reserving a small portion of traffic for random or exploratory recommendations provides unbiased interaction data. This prevents the system from locking into an increasingly narrow view of what users want.